Multi-Agent AI Lab: LLM Agents, Retrieval and Resource-Aware Cooperation

How does one agent decide?

Session 1 · Kickoff and theory

Julia Schaumeier

IN2106 · WS 2026/27 · 19 October 2026



Today

1. Welcome, and who is here
2. The Archive Games: the course and the semester
3. Your hopes and worries
4. What is an agent?
5. Four ways an agent can decide
6. Forming the teams
7. Your first team task: design an agent
8. What to take away

How to reach us

Contact

Dr. Julia Schaumeier
j.schaumeier@tum.de

Prof. Dr. Matthias Grabmair
matthias.grabmair@tum.de

The course

Materials and announcements
Moodle — course page

Questions
email, or the Moodle forum

Team meetings
from late November, arranged with each team

Who is here?

About half a minute each

- Your name and programme
- The AI you have built or studied so far

The Archive Games

The archive

The archive is a maze of **rooms** — topic clusters full of music knowledge. But it's messy, like real information:



ambiguous titles



duplicates



outdated claims



hallucinated citations



buried evidence

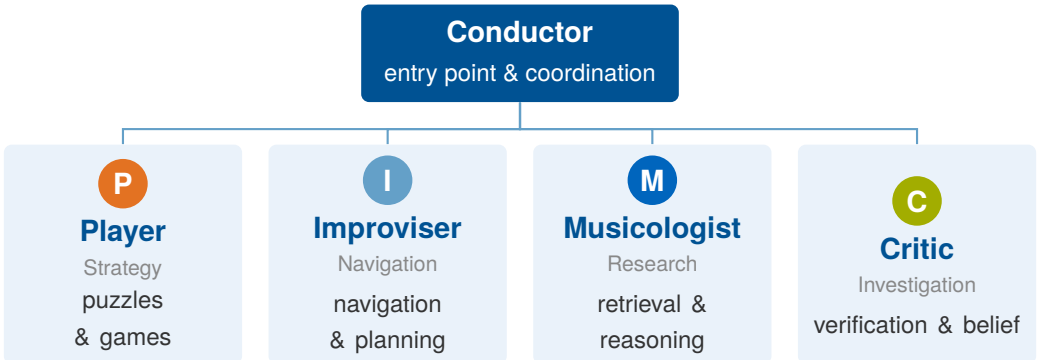
One chatbot can't cope.

So you send in a **team of specialised agents.**

Icons: alexdndz, Magnific, Uniconlabs, Elzicon, IconBaandar / Flaticon

Your Ensemble

Each student owns one specialisation; a **Conductor** directs the whole system.



The base quests

One each, served straight to its owner, so nobody waits on a teammate.

G

Gate

solve a puzzle to open a locked door

Player
Strategy

N

Night Round

collect every volume, with no map

Improviser
Navigation

R

Reading Desk

answer questions from the volumes in a room

Musicologist
Research

V

Variorum

decide what to believe, and why

Critic
Investigation

Harder quests, the Symphony and the tournament are set out in Session 5, after M2.

The semester in four blocks

Foundations & Design

- Kickoff + lectures
- Specialisation matching
- Lit. survey & design
- Presentation & QA

→ **M1**

Base build

- Build base quests
- Prototype integration
- Presentation + peer review

→ **M2**

Advanced build

- Hardening + new quests
- Experiments & analysis
- Presentation + peer review

→ **M3**

Tournament & Report

- Grand Tournament
- Final report

go Platinum

Dates

When	What
Mon 19 Oct, 2–3.30 pm	Session 1 — kickoff and theory
Mon 26 Oct, 2–3.30 pm	Session 2 — theory
Mon 2 Nov, 2–3.30 pm	Session 3 — theory
Mon 9 Nov, 2–3.30 pm	Session 4 — the Archive Games
Thu 12 Nov, 2 pm	M1 due — literature survey, agent design notes, architecture sketch, project plan
week of 16 Nov	M1 presentations and feedback, with each team
week of 30 Nov	M2 rehearsal — a trial run to check your code connects; not graded
Thu 10 Dec, 2 pm	M2 due — working prototype, base quest per specialisation, reflections
Mon 14 Dec, 2–3.30 pm	Session 5 — the advanced build
week of 14 Dec	M2 presentations and feedback, with each team; peer review (not graded)
21 Dec – 8 Jan	meeting break
week of 18 Jan	M3 rehearsal — the same, before M3; not graded
Thu 28 Jan, 2 pm	M3 due — final system, hardened quests
week of 1 Feb	final presentations and feedback on M3; the Grand Archive Tournament; peer review
Thu 4 Mar, 2 pm	Final report due

Mentor meetings with each team: weeks of 23 and 30 Nov; 7 Dec; 11, 18 and 25 Jan.

Grading & what's due

30%
M1 · design & literature
due Thu 12 Nov, 2 pm

You: literature survey; agent design note
Team: architecture sketch; project plan
The week after: literature-survey presentation, each your own part

30%
M2 · working prototype
due Thu 10 Dec, 2 pm

You: your base quest; a short reflection
Team: working prototype
The week after: progress presentation; peer review (not graded)

40%
M3 · final + tournament
due Thu 28 Jan, 2 pm

You: your hardened specialisation
Team: final system, experiments
The week after: final presentation and feedback; peer review
Thu 4 Mar, 2 pm: your final report

The same 100%, split by who earns it: ∼40% your own specialisation · ∼25% the team's integrated system · ∼30% (individual) presentations · ∼5% participation at peer review

Academic integrity

- Cite what you use: papers, ideas, figures and code. Code adapted from elsewhere names its source in a comment and keeps its licence
- Discuss ideas with other teams as much as you like; do not share code
- Your specialisation is your own work; the Conductor is the whole team's
- Report results as you measured them: never invent or adjust a number

Also read the Informatics Student Code of Conduct (2016) and the TUM University Library's citation practice guide.

Generative AI

Inside your agents	only the course model, through the course endpoint — every token is counted. No other model APIs in the submitted code
Writing code	allowed. You answer for every line and must be able to explain it when asked
Written work	allowed for language, structure and finding papers. The content, the analysis and the conclusions are yours. Check every citation against the source itself: language models invent references
Results	every number comes from your own runs and logs
Disclose	each deliverable ends with a short note: which tool, for what and what you checked. Undisclosed use counts as a breach of academic integrity

Hopes and worries

3 minutes — on your own

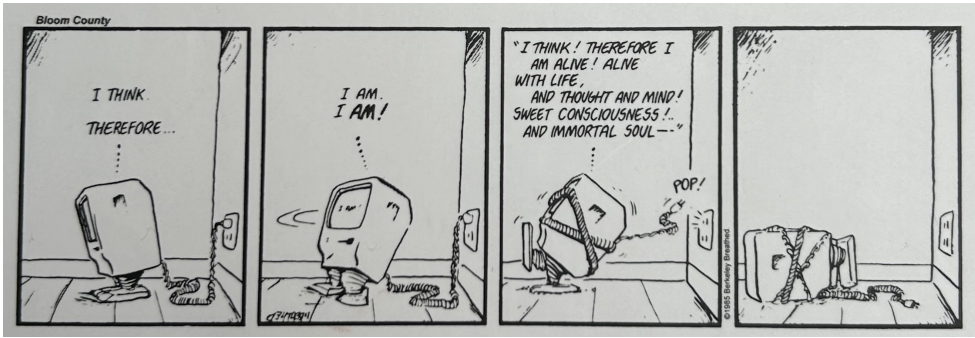
- Orange note: one thing you hope to learn
- Pink note: one thing you are worried about
- Stick them on the board

By the end of today, you will be able to

1. classify an agent's environment
2. tell an agent from ordinary software
3. write a performance measure, and spot how it can go wrong
4. compare ways of building an agent, and their limits
5. choose one for a new problem, and justify it

What is an agent?

Is it an agent?



Bloom County strip by Berkeley Breathed (1985), photographed from the cover of J. Pitt (Ed.), *The Computer After Me: Awareness and Self-Awareness in Autonomic Systems* (Imperial College Press, 2015)

Agent or not?

Hands up: agent — not an agent — depends

- a thermostat
- a spam filter
- a chess engine
- a robot vacuum
- a calculator
- a chatbot window

Situated, autonomous, flexible

An agent is a computer system situated in some environment, capable of autonomous action in that environment in order to achieve its delegated objectives.

An *intelligent* agent acts flexibly, which takes three properties:

- **Reactivity** — it responds to changes in its environment in time for the response to be useful
- **Proactiveness** — it pursues goals on its own initiative
- **Social ability** — it interacts with other agents, or with humans, to meet its objectives

Wooldridge (2009, Chapter 2)

Agents and objects

Object

- Encapsulates state and behaviour
- Has no control over its behaviour: `vacuum.clean()` runs whenever another object invokes it

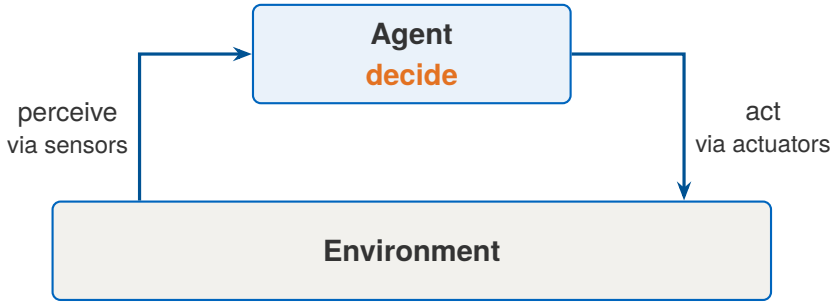
Agent

- Encapsulates state, behaviour *and the decision to act*
- Given the goal “keep the flat clean”, it decides whether, when and how to act — for example, to recharge first

Even a fixed rule (“clean when dirt is detected”) makes a system an agent: the system, not a caller, decides when to act.

Wooldridge (2009, Chapter 2)

The loop every agent runs



Everything in the next hour is a different way of filling the box in the middle: **how does this agent decide?**

Russell & Norvig (2021, Section 2.1)

Environments differ

Dimension	The question it asks
fully / partially observable	do the sensors give access to the complete relevant state?
deterministic / nondeterministic	is the next state fully determined by the current state and action?
static / dynamic	can the environment change while the agent deliberates?
episodic / sequential	does the current decision affect future ones?
discrete / continuous	are states, time and actions discrete?
single-agent / multi-agent	do other agents act in the environment?
known / unknown	does the agent know the environment's rules (its dynamics)?

Russell & Norvig (2021, Section 2.3)

Classify a task environment

4 minutes — in pairs, two tasks each

- cleaning a flat the vacuum robot has never seen
- playing chess against a clock
- playing a hand of poker
- solving a crossword
- driving a taxi through city traffic
- sorting post by its postcode

Each is a **task**, not a machine: what you classify is the world the task happens in.

Observable? Deterministic? Static? Episodic? Discrete? Single-agent? Known?










Photos via Wikimedia Commons: Auledas, Ajepbah (CC BY-SA 3.0), Santeri Viinamäki, Spaghettifier, Grendelkhan (CC BY-SA 4.0), Martin Vorel

Environments — possible answers

	Observable	Deterministic	Static	Episodic	Discrete	Agents
cleaning a new flat	partially	nondeterministic	dynamic	sequential	continuous	single
chess against a clock	fully	deterministic	semi-dynamic	sequential	discrete	multi
a hand of poker	partially	nondeterministic	static	sequential	discrete	multi
a crossword	fully	deterministic	static	sequential	discrete	single
driving a taxi	partially	nondeterministic	dynamic	sequential	continuous	multi
sorting post	fully	deterministic	static	episodic	discrete	single

All six are **known**: the rules are known, even where the state (the flat, the cards) is not.

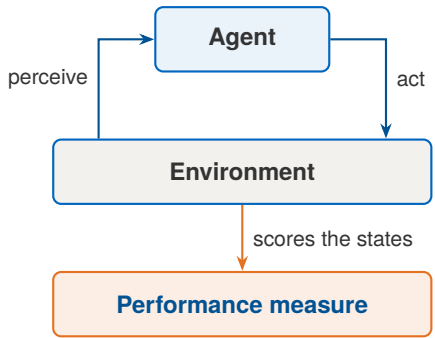
Where your answers differ, it is usually the **boundary** that differs: what you counted as part of the environment, and what you left outside it.

More examples: Russell & Norvig (2021, Figure 2.6)

Rational by what measure?

The question moves on: from the dimensions of the environment to doing well in it.

- Rationality depends on the **performance measure**, set by the designer
- Keep three things apart: what you *want*, what you *measure*, what the agent *maximises*. Rational means doing best by what you measure
- Rational $\neq$ omniscient: an agent that could not have known better has not acted irrationally



Russell & Norvig (2021, Section 2.2)

Score the robot vacuum

1 minute alone, then 2 minutes in pairs

- What should a robot vacuum be scored on?
- Name three things and say which matters most

Careful what you measure

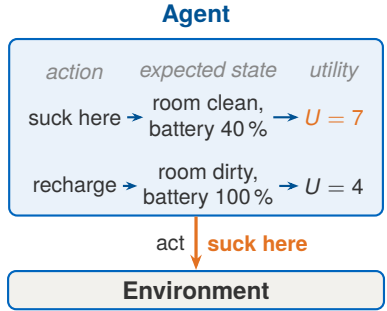
Measures what the robot does	Measures the state of the flat
Dirt collected. Best score: tip the dirt back onto the floor and collect it again.	*Dirt left on the floor at 6 pm.* Best score: a clean floor. There is no other way.

- **Specification gaming**: the agent meets the measure as written, not what the designer meant — what you *measure* misses what you *want*
- So measure the state you want the world in, not the behaviour you expect
- Which criteria for the robot vacuum could be gamed in this way? Area covered, time spent cleaning, no collisions, battery left …

Russell & Norvig (2021, Section 2.2.1); Krakovna et al. (2020)

Goals and utility

- A **goal** separates desired from undesired states: the flat is clean, or it is not
- A **utility function** U gives each state s a number, so the agent can trade one objective against another
- The robot vacuum's criteria, weighted, form a utility function: cleanliness v. battery, noise,…
- **Maximum expected utility**: choose the action a that maximises $\sum_s P(s \mid a)\, U(s)$



A utility function is the designer's attempt to give the agent the performance measure in a form it can use. Where the attempt misses, what the agent *maximises* is not what you *measure*: it maximises faithfully and still does the wrong thing.

Russell & Norvig (2021, Section 2.4 and Chapter 15)

Rational with limited resources

The question moves on: from the measure to the cost of answering.

- Deciding costs are: computing the answer, gathering necessary information, time spent not acting, etc.
- **Bounded rationality**: deciding well on the information, computation and time you actually have — a doctor in A&E treats the most likely cause on incomplete evidence, and without waiting for every test
- **Bounded optimality**: the best agent you can build for a given machine — speech recognition on a phone should be the best that fits its chip and battery, not the one that would win on a server

So there is no single best way to decide. The rest of today is four of them.

Russell & Subramanian (1995)

Human reactive agents

3 minutes — everyone stands

- Secretly pick two people in the room
- Keep moving so that you are the same distance from both

Four ways an agent can decide

Four paradigms

Reactive

Act on what you sense; no world model.
automatic gates · Pac-Man ghosts

Deliberative

Reason over internal model to plan towards a goal.
robot vacuums · job-shop scheduler

Learning

Improve from experience; explore then exploit.
trading bot · social-media feed

Layered

Paradigms combine — a real agent is often layered.

And one more kind: AI agents — an LLM as reasoning core, with tools and retrieval. Session 3 covers them.

Reactive agents

- **Condition–action rules** on what it perceives now (simple reflex agents)
- No internal state: no world model, no memory of what it perceived before
- **Subsumption architecture**: a hierarchy of behaviours in which the basic ones, such as avoiding obstacles, take precedence
- Examples: an automatic gate; a thermostat; the first Roomba (2002), with fixed behaviours such as wall-following and spiralling; the ghosts in Pac-Man

Russell & Norvig (2021, Section 2.4); Wooldridge (2009, Chapter 5)



Photo: Larry D. Moore, CC BY 4.0, via Wikimedia Commons



Photo: midnightcomm (Flickr), CC BY 2.0, via Wikimedia Commons

Reactive: the first Roomba



Photo: Larry D. Moore, CC BY 4.0, via Wikimedia Commons

- Senses bumps, cliffs and dirt
- Keeps nothing: every step starts afresh

```
while True:
    percept = sense()    # bumps, cliffs, dirt
    # decide: rules on what it senses now
    if percept.cliff:    action = "back up"
    elif percept.bumped: action = "turn away"
    elif percept.dirt:   action = "spiral here"
    else:                action = "drive on"
    act(action)
```

Reactive: strengths and limits

Strengths

- low latency
- low computational cost
- robust: degrades gracefully when parts fail

Limitations

- cannot use anything it does not perceive right now
- cannot plan ahead
- global behaviour emerges from rules, so it is hard to design for complex goals

A simple reactive agent makes a good **baseline**: it gives a lower bound against which more complex designs are measured.

Deliberative agents

- Keep a **world model** and **plan** over it: search over states towards a goal, or constraint satisfaction
- Example: a robot vacuum that maps the flat



Photo: Smart Home Perfected, CC BY 2.0, via Wikimedia Commons

BDI, a classic deliberative architecture:

Beliefs
what it thinks is true
the model it plans over, and it may be wrong

Desires
what it would like
more than it can have, so they compete

Intentions
what it has committed to
so it stops re-deciding at every step

Russell & Norvig (2021, Chapters 3, 6 and 12); Wooldridge (2009, Chapter 4)

Deliberative: a vacuum that maps



Photo: Smart Home Perfected, CC BY 2.0, via Wikimedia Commons

- Builds a map with its lidar
- Commits to a room and keeps to its plan until the map says the plan no longer works

```
beliefs = Map()          # its picture of the flat
desires = ["clean every room", "keep the battery up"]
intention = None         # committed to nothing yet
plan = []                # the steps still to take
while True:
    percept = sense()  # lidar, position, battery
    beliefs.update(percept)
    # deliberation: what to achieve
    if intention is None or beliefs.finished(intention):
        intention = choose(desires, beliefs)  # kitchen
        plan = route(beliefs, intention)      # search
    # means-ends: how; replan only if the plan fails
    elif not plan or not beliefs.allows(plan):
        plan = route(beliefs, intention)
    action = plan.pop(0)
    act(action)
```

Deliberative: strengths and limits

Strengths

- plans ahead, towards goals that take many steps
- can explain what it does: the plan, and the beliefs behind it
- one model serves many goals

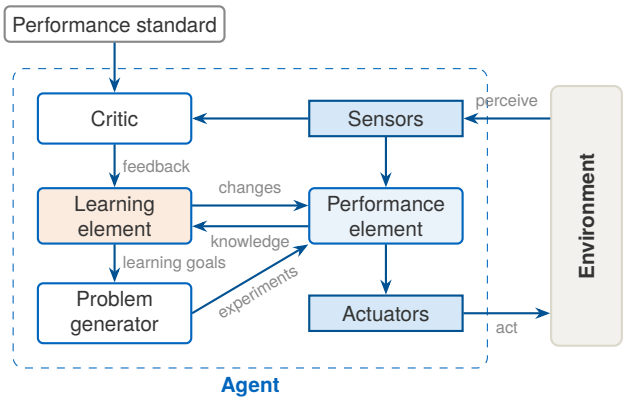
Limitations

- only as good as its world model
- planning takes time; the world may change meanwhile
- it cannot plan a place it has not seen: it must explore first

Commitment has to be tuned: holding to a plan is what stops endless re-deciding, and what keeps a stale plan running.

Russell & Norvig (2021, Section 4.5); Wooldridge (2009, Chapter 4)

Learning agents



after Russell & Norvig (2021, Figure 2.15)

- The **performance element** decides what to do
- The **critic** judges how well that went, against a fixed performance standard
- The **learning element** improves the performance element
- The **problem generator** suggests something new to try
- Examples: trading bots; social-media feeds

Learning: a vacuum that learns



Photo: Auledas, CC BY-SA 4.0, via Wikimedia Commons

- Later Roombas, such as the i7, learn which rooms get dirtiest and when to clean
- The critic judges each step; the agent does more of what went well

```
learned = {}            # how well choices went
while True:
    percept = sense()
    # decide: best so far, sometimes something new
    if random() < 0.1:
        action = something_new()    # problem generator
    else:                           # performance element
        action = best(learned, percept)
    act(action)
    score = critic(sense())         # vs. the standard
    # learning element:
    learn(learned, percept, action, score)
```

Learning: strengths and limits

Strengths

- improves with experience
- copes with what its designer did not foresee
- adapts as the world or its users change

Limitations

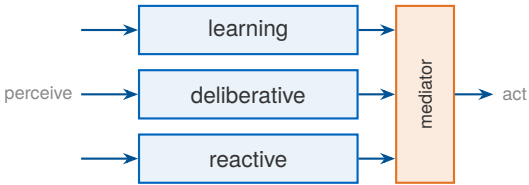
- needs experience, and early mistakes cost
- what it learnt is tied to the world it learnt in, and that world moves
- it learns what the performance standard scores, which may not be what you want

Explore or exploit has to be balanced: try something uncertain, or stay with the best so far. The simplest case is the multi-armed bandit.

Sutton & Barto (2018, Chapter 2)

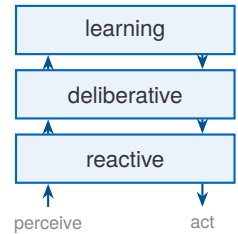
Layered architectures

Horizontal



each layer sees what is perceived, proposes what to do

Vertical



what is perceived climbs; what to do comes back down

A typical stack: a reactive layer for immediate responses, a deliberative layer for planning, a learning layer that tunes both.

after Wooldridge (2009, Chapter 5)

Layered: a modern robot vacuum



Photo: Smart Home Perfected, CC BY 2.0, via Wikimedia Commons

- A reflex, a planner and learning, stacked
- **Vertical**: only the bottom layer senses and acts; it asks up when it has nothing to do

```
while True:
    percept = sense()                # the bottom layer senses
    # reactive layer
    if percept.cliff or percept.bumped or percept.dirt:
        action = reflex(percept)     # the rules from before
    else:                            # nothing to do: ask up
        # deliberative layer: the BDI loop, without act
        beliefs.update(percept)
        if intention is None or beliefs.finished(intention):
            # learning layer: which room next
            score = critic(beliefs) # how the last room went
            learn(learned, beliefs, intention, score)
            intention = best(learned, beliefs)
            plan = route(beliefs, intention)
        elif not plan or not beliefs.allows(plan):
            plan = route(beliefs, intention)
        action = plan.pop(0)         # back down
    act(action)                      # the bottom layer acts
```

Layered: strengths and limits

Strengths

- reacts fast and plans ahead at the same time
- each layer uses the paradigm that suits its job
- layers can be built and tested one at a time

Limitations

- horizontal: the mediator handles every conflict and can become the bottleneck, but a failing layer only stops proposing
- vertical: nothing covers for a failing layer, so the agent drops to what the layers below it can do
- every layer can pass its tests while the agent fails: the faults are in the interaction

Wooldridge (2009, Chapter 5)

Forming the teams

5 minutes

- Form teams of three or four, pin your names on the board
- Then sit together, your first team task comes next

Your first team task

9 minutes — in your new team

Design a delivery drone four ways — reactive, BDI, learning, layered.

- What does each one perceive, what does it keep, and how does it decide?
- Where would each one fail?
- Which would you build first, as a baseline?



Photo: Roksenhorn, CC BY-SA 4.0, via Wikimedia Commons

The drone — one possible answer

	Perceives	Keeps	Decides	Fails when
Reactive	position, bearing, obstacles, battery	nothing	obstacle ahead → climb; battery low → land; else fly on	a concave obstacle traps it; no-fly zones are invisible; no reserve to return
BDI	the same	beliefs: map, no-fly zones, weather; desires: deliver, return; intentions: the route	plans, commits, replans when a belief changes	the map is wrong (a new crane); replanning too slow; over-committed when the wind turns
Learning	the same	past flights: energy per route and wind, landing spots that worked	picks routes by learned estimates, explores now and then	too little experience; exploring costs a crash; the weather drifts
Layered	the same	all of the above	collision avoidance overrides; deliberative routing; learning tunes the energy model	the faults are in the interaction, not the layers

Baseline first: the reactive drone — cheap, and the lower bound every other design must beat.

What to take away

- What a good design is depends on the **environment** — seven dimensions to classify it
- An **agent** is situated and autonomous: it decides when to act, not a caller
- **Rational** means best expected performance by a measure you set — and a measure can be met by behaviour you never wanted
- Four ways to decide: **reactive**, **deliberative (BDI)**, **learning**, **layered** — each with its own limits
- Choose between them by what the agent can **observe** and what its decisions **cost**: a correct answer that costs too much is not a good answer

Three quick questions

Hands up

1. Is a thermostat an agent — and if so, which paradigm?
2. A satnav that re-routes after a missed turn: which paradigm, and why?
3. Name a performance measure an agent could satisfy while doing something you never wanted.

One-minute paper

1 minute — on your own

- Orange note: the clearest point today
- Pink note: the muddiest point today
- Leave them on the wall as you go

If you want to read more

If today's topics are new to you, or you want the full story:

Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). Wiley. — Chapters 1–2 and 4–5

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed., global ed.). Pearson. — Chapter 2; Chapters 3 and 6 for search and constraints

Rao, A. S., & Georgeff, M. P. (1995). BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, 312–319. AAAI Press.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. — Chapter 2, for learning

Choosing a specialisation

P Player Strategy

Gate. Your agent starts at a locked door that describes a puzzle in words. Solve it to open the door, within a limited number of attempts.

I Improviser Navigation

Night Round. Your agent explores rooms it has no map of, collects every volume and brings it to the entrance. The trolley holds only so many.

M Musicologist Research

Reading Desk. Your agent answers questions from one room's volumes and names the ones that support each answer.

C Critic Investigation

Variorum. Your agent decides what to believe, and why. Teams of four only.

Until we meet again

- **By Mon 26 Oct**: agree in your team who takes which specialisation — Session 2 starts with it
- **If you like**: start reading up on agents and their environments — it will be useful for your literature survey
- **Mon 26 Oct**, Session 2: how do several agents decide together? With the M1 brief, the reading list and the API reference to start your sketch

References

References

Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Kenton, Z., Leike, J., & Legg, S. (2020). *Specification gaming: The flip side of AI ingenuity*. DeepMind blog, 21 April 2020.

Rao, A. S., & Georgeff, M. P. (1995). BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, 312–319. AAAI Press.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed., global ed.). Pearson.

Russell, S. J., & Subramanian, D. (1995). Provably bounded-optimal agents. *Journal of Artificial Intelligence Research*, 2, 575–609.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). Wiley.